PRODUCT AND TECHNICAL GUIDE

E-commerce ROI Dashboard

A project document explaining the product, metrics, data contract, architecture, workflows, and implementation details.

Streamlit application | Python, pandas, Plotly

Prepared October 2026

1. Project overview

The dashboard brings paid media cost and ecommerce outcomes into one interactive view. Marketers and store operators can compare spend with attributed revenue and orders, inspect channel and campaign performance, and explore daily trends using a common filter set.

Primary goals

- Understand overall spend, revenue, orders, ROAS, and ROI after ad spend for a selected period.
- Compare paid channels and campaigns using aggregate ratios rather than averages of row-level ratios.
- Identify campaigns that may need review or may merit more investigation based on configurable ROAS and spend thresholds.
- Download normalized or filtered results for follow-up analysis.

What the product includes

Experience	What it provides
Shared exploration	Date, platform, campaign, product, region, device, and single-currency filters.
Performance analysis	KPIs, prior-period comparison, channel and campaign breakdowns, daily trend series.
Decision support	Session-level budget and revenue targets, ROAS target, pacing, and campaign opportunity flags.
Data operations	CSV upload, schema and quality feedback, normalized preview, and CSV exports.

The checked-in sample values are synthetic and illustrate how the calculations work. They do not represent actual store results.

2. How to use the dashboard

Start on the Dashboard page, choose the reporting window and dimensions in the sidebar, set targets if needed, then use the other pages to compare or inspect the filtered data.

Pages

Page	Purpose
Dashboard	Core KPIs, previous-period comparison, target pacing, campaign opportunities, daily trend, and filtered rows.
Overview	High-level KPIs and a daily trend for the shared filter selection.
Channel comparison	Compare spend, revenue, orders, clicks, ROAS, or ROI by advertising platform.
Campaign and business breakdowns	Group and sort by campaign, product, region, device, or platform and campaign.
E-commerce trends	Select metric series to display by day; ratio series are derived from daily totals.

Shared filters

- The date range is inclusive. Defaults cover all dates available in the loaded data.
- Platform, campaign, product, region, and device selections carry across pages.
- The currency control selects one currency at a time. The app does not exchange or combine currencies.
- If a filter selection returns no rows, the page reports an empty result.

Targets and opportunities

Target controls are available in the Dashboard sidebar and persist for the current Streamlit session only. Budget pacing compares spend to expected budget use based on elapsed calendar days in the selected range. Revenue goal progress uses revenue to date. The opportunity list flags high-spend campaigns below target ROAS and low-spend campaigns at or above it. Treat these flags as review prompts, not automatic budget recommendations.

3. Metrics and interpretation

Monetary values are shown in the selected currency. The dashboard calculates ratios from totals for the active selection or group; it does not average daily or campaign ratios.

Core ecommerce metrics

Metric	Formula	Interpretation
Spend	Sum of spend	Paid media cost in the selected data.
Revenue	Sum of revenue	Attributed ecommerce revenue supplied by the CSV.
Orders	Sum of orders	Attributed orders. Legacy `conversions` input is accepted as an alias.
ROAS	Revenue / spend	Revenue per currency unit of ad spend; displayed as a multiple.
ROI after ad spend	(Revenue - spend) / spend	Revenue return after ad spend only; displayed as a percentage.
Average order value	Revenue / orders	Average attributed revenue per order.
Cost per order	Spend / orders	Average ad spend per attributed order.
Click-to-order rate	Orders / clicks	Share of clicks that resulted in an order.

Optional contribution estimate

When product_cost and shipping_cost are complete for all selected rows, contribution profit after ads is revenue - refunds - discounts - product_cost - shipping_cost - spend. Omitted refunds and discounts are treated as zero. Contribution margin divides this estimate by revenue - refunds - discounts.

This contribution estimate excludes taxes, payment fees, and operating overhead. It is not accounting profit. ROI remains the separate revenue-minus-ad-spend ratio.

Undefined ratios

If a denominator is zero, the relevant ratio is displayed as unavailable. This avoids suggesting that a zero denominator produces a valid zero result.

4. CSV data contract

The input is a campaign-level CSV. The intended grain is one row per date, platform, and campaign. If non-identical rows repeat that key, the app retains and flags them for review rather than discarding values.

Required columns

Column	Description
date	Reporting date parseable by pandas.
platform	Advertising platform, such as Google Ads or Meta Ads.
campaign	Campaign name.
spend	Numeric ad spend.
impressions	Numeric impression count.
clicks	Numeric click count.
orders	Numeric attributed ecommerce order count.
revenue	Numeric attributed ecommerce revenue.

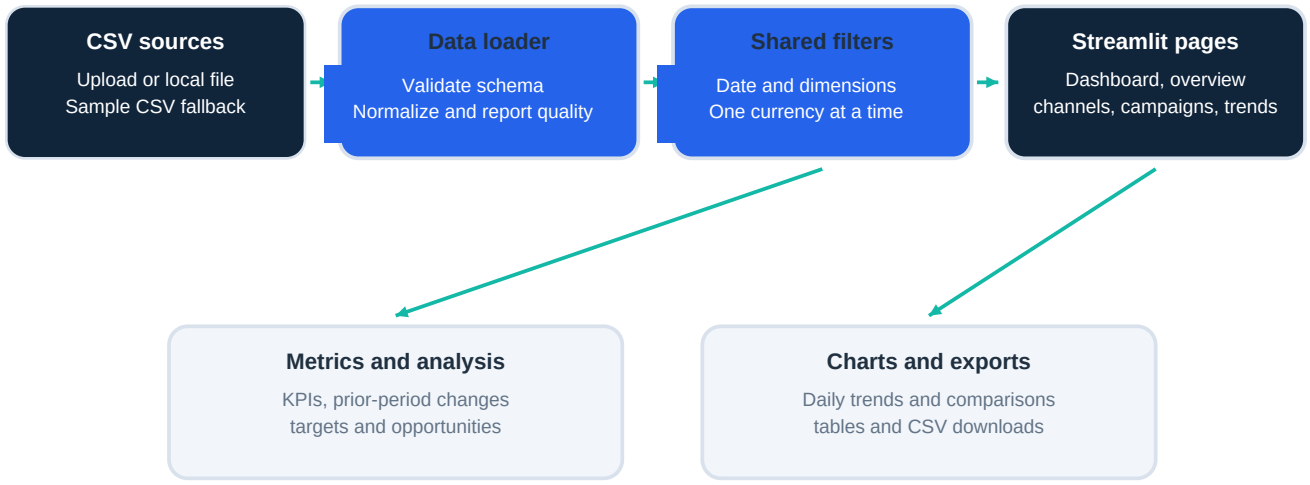
Optional columns

Column	Use
currency	ISO code for monetary data. Missing values default to USD with a quality notice.
product	Product grouping; missing column defaults to All products.
region / device	Optional dimensions; missing values default to Unknown.
product_cost	Product cost / cost of goods sold used in contribution profit.
refunds / discounts	Amounts subtracted from revenue for contribution estimates; omitted values default to zero.
shipping_cost	Fulfillment cost used in contribution profit.

Keep spend and revenue in the same currency and scale. The loader also supports the legacy conversions column as an alias for orders. Dates are treated as calendar dates; the file does not define a reporting timezone or attribution window.

5. Application architecture

The application separates Streamlit page layout from reusable data and calculation helpers. Each page uses a common data-loading and filter path so a selected file and shared dimensions are handled consistently.



Request and data flow

- app.py configures Streamlit navigation and routes to one of five pages.
- src/data_ui.py provides the shared CSV uploader, quality panel, normalized preview, and data download.
- src/data_loader.py reads the upload or local/default CSV, validates required fields, normalizes data, and derives row-level calculations.
- src/filters.py maintains shared date and dimension selections, including a single currency.
- src/metrics.py calculates totals, grouped ratios, prior-period comparisons, opportunity flags, and contribution estimates.
- src/charts.py builds daily trend and platform comparison figures from aggregated data.

A filter update triggers a Streamlit rerun. The page reloads the selected data source, applies the stored filter choices, recomputes its result, and renders the updated view.

6. Data quality and exports

Expand Data quality and export in the sidebar to review import diagnostics and download normalized rows.

Quality handling

- Exact duplicate rows are removed and counted.
- Rows with invalid dates are dropped and counted. If none remain, the app shows a readable error.
- Missing or nonnumeric required numeric values are coerced to zero and reported by field.
- Repeated date/platform/campaign keys with differing values are counted and retained for review.
- Missing optional dimensions receive documented defaults; missing currency values default to USD and are called out.
- Incomplete product_cost or shipping_cost values suppress contribution profit for the affected selection or group.

Available downloads

Location	Download
Sidebar	Normalized cleaned campaign data.
Dashboard	Filtered rows and campaign opportunity list.
Channel comparison	Aggregated platform summary.
Campaign/business breakdowns	Selected grouping and derived metrics.
Charts	Plotly toolbar image export.

Data review

Before relying on results, inspect the cleaned preview and warnings, verify currency and revenue definitions, and resolve duplicate campaign-day keys with the source system. The app does not infer an attribution model or repair inconsistent source records.

7. Project structure and operating notes

Path	Responsibility
app.py	Streamlit navigation entry point.
pages/	Dashboard, overview, channel, breakdown, and trend pages.
src/data_loader.py	CSV validation, normalization, quality report, derived row metrics.
src/data_ui.py	Shared upload, diagnostics, preview, downloads, and target inputs.
src/filters.py	Shared UI filters and reusable row filtering.
src/metrics.py	Aggregate metrics, grouping, comparisons, and campaign opportunities.
src/charts.py	Daily and channel visualizations.
tests/	Automated checks for metrics, filtering, schema handling, comparisons, and chart selection.
README.md / PRODUCT_SPEC.md	Setup, architecture, intended behavior, and acceptance criteria.

Run locally

Create and activate a virtual environment, install requirements.txt, then run `streamlit run app.py`. Run automated checks with `python -m unittest discover -s tests`.

Important unresolved decisions

- Confirm the source currency and whether amounts use major or minor units.
- Document the attribution model and attribution window for orders and revenue.
- Confirm whether revenue is gross or net of discounts, refunds, taxes, and shipping.
- Confirm reporting timezone and whether dates come from the ad platform or store system.
- Review whether the prototype contribution formula should include additional costs or different tax treatment.

Until confirmed, interpret ROI as (revenue - ad spend) / ad spend. Treat contribution profit as a configurable estimate, not an accounting result.